INFORME DE LA FASE DE ANÁLISIS Y DISEÑO

Samuel Mauricio Bastidas Jaramillo (1138525766)

Sergio Alejandro Cabrers Narvaez (1086922688)

1. CONTEXTUALIZACIÓN Y ANÁLISIS DEL PROBLEMA

Se debe crear una red social llamada “Ude@Book” en el lenguaje de programación C++ haciendo uso de la programación orientada a objetos y el manejo de memoria dinámica en el entorno Qt, además se deben respetar las limitaciones impuestas.

El núcleo de Ude@Book lo constituyen sus usuarios, sobre quienes recae toda la dinámica relacional. Las reglas definen que las relaciones de amistad son estrictamente simétricas y consensuadas: cada usuario cuenta con una colección de cero o más amigos registrados en la plataforma. En cuanto a la gestión multimedia, los usuarios pueden estructurar sus fotografías en álbumes con tres niveles estrictos de privacidad: público (accesible para cualquier miembro de la plataforma), amigos (restringido únicamente a sus contactos directos) y privado (exclusivo para el propietario). La mensajería exige un control cronológico basado en fechas de emisión, administrando estados de lectura (leído o no leído). Asimismo, el sistema incorpora una forma de sugerencias de amistad que analiza la cercanía entre miembros de la red mediante la cantidad de amigos en común.

La visualización de imágenes y fotografías de perfil se abstrae mediante la impresión de rutas absolutas estructuradas bajo notación de sistemas de archivos Linux.

2. ARQUITECTURA

Para dar cumplimiento a la restricción que prohíbe el uso de la Standard Template Library (STL) y librerías externas de terceros para resolver aspectos esenciales, el sistema se fundamenta en estructuras de datos de autoría propia gestionadas mediante memoria dinámica.

En primer lugar, se diseñara la estructura propia Lista<T>, una plantilla de lista enlazada simple asociada a una clase propia Nodo<T>. Frente a los arreglos estáticos tradicionales, la lista enlazada representa la alternativa idónea para modelar entidades en una red social, donde colecciones como amigos, mensajes, álbumes y solicitudes varían continuamente de tamaño durante el ciclo de vida de la aplicación. Un arreglo estático impondría límites fijos de capacidad o desperdiciaría memoria preasignada; en contraste, la lista enlazada asigna y libera nodos bajo demanda en el montículo (heap), manteniendo un consumo óptimo de memoria. Al estar parametrizada como plantilla, la misma estructura reutiliza su lógica para gestionar punteros a usuarios, álbumes, fotos o mensajes, asegurando alta reutilización de código.

En segundo lugar, para la gestión y consulta de las relaciones de amistad, se optó por implementar la clase GrafoAmistad, cuya estructura interna es una matriz de adyacencia dinámica de booleanos. En Ude@Book, verificar si dos usuarios son amigos y calcular amigos en común son las operaciones más recurrentes. Mientras que recorrer listas enlazadas cruzadas representaría una complejidad temporal lineal O(N), la matriz de adyacencia permite consultar la arista entre dos nodos en tiempo constante O(1). La

matriz reserva memoria según la capacidad total de usuarios registrados, compensando el espacio ocupado con una reducción drástica del tiempo de cálculo en las sugerencias y filtros algorítmicos.

Finalmente, la arquitectura incorpora la clase transversal MonitorRecursos. Su propósito es centralizar los recursos que uso el sistema sin mezclar código de medición con las reglas de negocio. El monitor actúa como un registrador que cuantifica los bytes asignados y liberados en memoria dinámica, así como las iteraciones directas e indirectas consumidas por cada operación, garantizando el cumplimiento de los requerimientos de eficiencia exigidos.

3. DESCRIPCIÓN DEL DIAGRAMA DE CLASES

El diagrama de clases define la interacción entre los componentes del sistema, organizados bajo principios de alta cohesión y bajo acoplamiento:

RedSocial: Es la clase controladora central y el punto de entrada de la aplicación. Administra una colección Lista<Usuario*> con los usuarios de la plataforma y mantiene una relación de composición con GrafoAmistad. Esta clase coordina la autenticación de credenciales, el registro en memoria y la invocación de algoritmos globales como el cálculo de sugerencias de amistad.

Usuario: Es la entidad principal del modelo. Gestiona atributos personales mediante punteros a caracteres (char*) para nombre de usuario, contraseña, correo, fecha de nacimiento, ciudad, país, fecha de inscripción y la ruta de la foto de perfil. Contiene colecciones basadas en la plantilla propia: Lista<Usuario*> para amigos directos, Lista<Album*> para sus álbumes, listas de mensajes enviados y recibidos, y una lista de solicitudes de amistad. Incorpora la sobrecarga del operador de igualdad (operator==) para comparar instancias a partir de identificadores únicos en búsquedas internas.

Album y Foto: El contenido multimedia se organiza en una relación de composición jerárquica. Usuario compone álbumes, y cada Album compone un conjunto de objetos Foto a través de una Lista<Foto*>. Album define la visibilidad (público, amigos, privado), mientras que Foto almacena el tamaño en bytes, fecha de captura, contador de likes y la ruta absoluta en formato Linux.

Mensaje y SolicitudAmistad: Las comunicaciones e interacciones pendientes se modelan como entidades transaccionales. Mensaje registra marcas de tiempo, remitente, destinatario, texto y un atributo booleano (leido) para controlar el estado de lectura. SolicitudAmistad vincula un emisor con un receptor mediante un estado numérico que determina si la petición está pendiente, aceptada o rechazada, antes de consolidar la relación en el grafo.

GrafoAmistad: Modela la topología no dirigida de la red social mediante una matriz de adyacencia dinámica (bool**). Proporciona primitivas de alta velocidad para agregar aristas de amistad bidireccionales, verificar adyacencia en tiempo O(1) y calcular la cantidad de amigos en común evaluando la intersección lógica de filas.

Lista<T> y Nodo<T>: La base operativa de estas colecciones es la plantilla Lista<T>, que se apoya en Nodo<T>. La lista gestiona la inserción y eliminación dinámica de elementos, e implementa la sobrecarga del operador de indexación (operator[]) para facilitar el acceso ordenado a los elementos y para preservar la encapsulación sin afectar el rendimiento, la clase Lista<T> se define como clase amiga (friend) de Nodo<T>, lo que le permite manipular directamente los punteros internos de enlace sin exponerlos hacia el resto del sistema.

MonitorRecursos: Componente transversal de auditoría. Monitorea y reporta en tiempo real los bytes asignados y liberados en el montículo mediante el acumulador de memoria dinámica activa, así como el conteo de iteraciones ejecutadas en cada algoritmo antes de reiniciar el ciclo de interacción.

4. LÓGICA DE ALTO NIVEL DE ALGORITMOS ESENCIALES

a. Selección aleatoria ponderada de fotos de perfil al iniciar sesión:
Al iniciar sesión, el sistema muestra tres fotos de perfil de amigos seleccionadas de forma aleatoria ponderada según la afinidad en la red:

1.El algoritmo toma el identificador del usuario autenticado y consulta el GrafoAmistad para determinar los amigos en común que comparte con cada uno de sus amigos directos.

2.Se asignan pesos relativos según la coincidencia: peso 1 para 10 o menos amigos en común; peso 2.5 para un rango entre 11 y 20; y peso 3.5 para más de 20 amigos en común.

3.Se construye un rango probabilístico acumulativo proporcional a dichos pesos.

4.Mediante el generador aleatorio, se extrae un valor dentro del rango total para seleccionar al candidato correspondiente. El proceso se repite hasta obtener tres amigos distintos, descartando duplicados.

5.Se imprimen en consola las rutas absolutas de las tres fotos de perfil seleccionadas. El sistema activa una bandera de control que autoriza al usuario a alternar este grupo por otro conjunto no repetido una única vez durante la sesión.

b. Sugerencias de amistad:
Este proceso determina los contactos más pertinentes explorando la red de segundo orden:

1.Se valida que el parámetro entero K esté dentro del rango permitido (1 a 20).

2.Se recorre la matriz de adyacencia del GrafoAmistad identificando los amigos de los amigos del usuario actual, descartando al propio usuario y a quienes ya pertenezcan a su lista de amigos.

3.Para cada candidato elegible, se calcula la cantidad exacta de amigos en común evaluando la intersección lógica de sus vectores de adyacencia.

4.Los candidatos se ordenan de forma descendente según su cantidad de amigos en común utilizando un algoritmo de ordenamiento interno por inserción adaptado a la lista.

5.Se seleccionan las primeras K posiciones de la lista ordenada y se imprimen en consola con la opción de remitirles una solicitud de amistad.

c. Navegación en carrusel de fotos:
Permite recorrer de forma interactiva y circular el contenido de un álbum:

1.Una vez validada la privacidad del álbum frente al visitante, se recupera su Lista<Foto*> y se sitúa un índice en la posición inicial (cero).

2.Se muestra en consola la ruta absoluta de la foto activa y su cantidad de likes.

3.El sistema lee la orden ingresada: si se solicita "siguiente", el índice avanza mediante aritmética modular: (indice + 1) % total_fotos; si se solicita "previa", se retrocede aplicando (indice - 1 + total_fotos) % total_fotos.

4.Si se ingresa la opción de like, se invoca el método correspondiente de la foto para alternar el valor en memoria.

5.El ciclo se repite de manera continua hasta que el usuario elija la opción de salir.

d. Filtrado de mensajes por rango de fechas:
Permite recuperar mensajes recibidos dentro de un intervalo temporal definido sin alteral su estado:

1.El algoritmo recibe la fecha inicial y la fecha final, normalizándolas a marcas de tiempo comparables.

2.Se recorre secuencialmente la lista mensajesRecibidos del usuario.

3.Para cada nodo se extrae el objeto Mensaje y se comprueba si su timestamp se encuentra dentro del rango (ambos extremos incluidos).

4.Aquellos mensajes que satisfacen la condición se imprimen en pantalla indicando emisor, fecha y contenido.

5. Se mantiene sin cambios el estado del atributo leído de cada mensaje, ya que esta funcionalidad es una consulta nada más(a diferencia de la opción "Leer nuevos mensajes", la cual sí transiciona los mensajes no leídos a leídos).

e. Medición de consumo de recursos:
El seguimiento de eficiencia opera de manera transversal mediante MonitorRecursos:

1.En cada operación de reserva de memoria (new) ejecutada en listas, nodos o entidades, se notifica al monitor el número de bytes asignados con el operador sizeof y se añade al acumulador.

2.De igual forma, al invocar un destructor (delete), se descuentan los bytes liberados, manteniendo la métrica de memoria activa.

3.En los bucles de recorrido, ordenamiento y búsqueda, se colocan invocaciones que incrementan el contador de iteraciones directas e indirectas.

4.Al concluir cualquier funcionalidad del menú, se imprimen las métricas registradas y se reinicia el contador de iteraciones para la siguiente instrucción.

5. FORMATO DE PERSISTENCIA EN ALMACENAMIENTO PERMANENTE Y ESTRATEGIA DE RETENCIÓN

De acuerdo con las pautas del desafío, el número de archivos a utilizar, su estructura interna, así como la periodicidad y la estrategia de guardado y actualización de los datos en el almacenamiento permanente quedan bajo el criterio y diseño del equipo desarrollador. Esta arquitectura de persistencia se concibió balanceando el criterio de eficiencia en operaciones de entrada/salida y el guardado de los datos.

Para evitar el costo computacional de un archivo monolítico donde cualquier cambio mínimo obligaría a reescribir toda la base de datos en disco con un costo O(N) inadmisible, se optó por un esquema modular de archivos planos de texto (.txt) con delimitadores por tubería (|):

1.usuarios.txt: Datos de cuenta y perfil. Estructura:
ID_Usuario|Username|Clave|Email|Fecha_Nacimiento|Ciudad|Pais|Fecha_Inscripcion|Ruta_Foto_Perfil

2.amistades.txt: Pares de adyacencia simétrica para reconstruir el grafo en el arranque. Estructura:
ID_Usuario_A|ID_Usuario_B

3.albumes_fotos.txt: Relación de álbumes y fotografías. Estructura:
ID_Album|ID_Propietario|Nombre_Album|Privacidad|ID_Foto|Ruta_Linux|Tamano_Bytes|Fecha_Captura|Likes

4.mensajes.txt: Registro histórico de comunicaciones. Estructura:
ID_Mensaje|ID_Emisor|ID_Receptor|Contenido|Timestamp|Leido

5.solicitudes.txt: Transición de solicitudes pendientes o resueltas. Estructura:
ID_Solicitud|ID_Emisor|ID_Receptor|Estado

La estrategia de actualización y retención adopta un modelo híbrido orientado a eventos clave y cierre ordenado:

Persistencia por eventos críticos: Cuando ocurre una mutación de estado volátil (el envío de un nuevo mensaje, el cambio de estado de un mensaje a leído, la emisión o aceptación de una solicitud de amistad, o la alternancia de un like), el sistema no posterga la escritura indefinidamente. En su lugar, aplica una persistencia dirigida actualizando únicamente el archivo. Esto asegura que la información permanezca resguardada ante interrupciones abruptas de la sesión o salidas inesperadas del flujo, evitando pérdidas de datos.

Sincronización consolidada al cierre: Al cerrar sesión o salir formalmente de la plataforma, el sistema ejecuta una verificación general de estado, sincronizando cualquier estructura pendiente antes de liberar la memoria de trabajo.

Eficiencia e integridad: Para mantener la eficiencia, los archivos con datos puramente informativos (como álbumes o perfiles base) no se reescriben si no han sufrido modificaciones. Además, las reescrituras se apoyan en archivos temporales intermedios que se renombran al finalizar la escritura, impidiendo que una operación incompleta dañe el registro permanente.

En el arranque del programa, la carga se realiza en lectura secuencial: primero se cargan los usuarios para dimensionar la matriz, luego se leen las amistades para poblar el grafo, y finalmente se incorporan los álbumes, mensajes y solicitudes a las listas internas.

6. DIFICULTADES DE DISEÑO Y CONSIDERACIONES DE IMPLEMENTACIÓN

Durante la etapa de diseño se identificaron algunos retos derivados de las restricciones de implementación y la exigencia de optimización:

Gestión manual de memoria en cadenas: El primer desafío radica en la manipulación manual de cadenas de caracteres mediante punteros char*, dada la prohibición de la clase string de la STL. Esto exige implementar de forma estricta constructores de copia profunda y sobrecargas del operador de asignación en clases como Usuario, Mensaje y Foto. Una copia superficial provocaría que múltiples objetos compartieran punteros hacia las mismas posiciones del montículo, causando accesos a memoria inválida o liberaciones dobles al destruir los objetos.

Destrucción segura de estructuras enlazadas: Para evitar fugas de memoria, la clase Lista<T> debe contar con un destructor iterativo que recorra nodo por nodo desasignando la memoria de forma ordenada. Se descartó una destrucción puramente recursiva en las listas para prevenir desbordamientos de la pila de ejecución (stack overflow) al procesar miles de registros. De manera complementaria, el destructor de Usuario debe vaciar ordenadamente sus listas internas de álbumes, mensajes y solicitudes antes de destruirse, asegurando que no quede memoria huérfana en el sistema.

Paso por referencia constante: Una consideración de diseño determinante es la política de paso por referencia constante (const Tipo&) en parámetros de métodos y funciones. El paso de estructuras complejas por valor desencadenaría copias masivas e innecesarias en el montículo, distorsionando las mediciones de memoria e iteraciones del MonitorRecursos y degradando el desempeño del sistema.

Organización modular (.h y .cpp): Por último, el proyecto se organizará bajo una separación modular: las declaraciones de clases y prototipos residirán en archivos de cabecera (.h), mientras que la lógica de métodos se desarrollará en archivos de implementación (.cpp). En el caso de las plantillas genéricas Lista<T> y Nodo<T>, sus definiciones se mantendrán en las cabeceras correspondientes, garantizando su correcta compilación y compatibilidad con las directivas del compilador de C++.